

AI 100 講 — 補充單元

Token 經濟學

從敢花到會花的思維轉換

AI 時代的「油錢」管理

按 → 或空白鍵開始

今天的旅程

1

Token 是什麼？

AI 世界的貨幣單位

2

Phase 1：敢花 Token

黃仁勳的 50% 法則 — 先用起來

3

Phase 2：會花 Token

MCP 爭議揭示的效率革命

4

工具架構演進

CLI → API → MCP → Agent OS

5

你的學習路徑升級

從用工具到 Token 管理思維

回顧：你的 AI 學習旅程



但是...

學會開車（用工具）、選了好車（選工具）、甚至改裝了車（創工具）

你有沒有想過「油錢」怎麼花最值得？

Part 1

Token 是什麼？

AI 世界的貨幣單位

Token = AI 的「字」

你輸入的每一個字

□□□□□□□□

□□ ≈ 1.5~2 token / □

□□□ ≈ **14 tokens**

你打字 = 花錢

AI 回覆 = 也花錢

生活比喻

Token 就像手機的「通話分鐘數」

你講話 = 花分鐘數 (輸入 token)

對方回答 = 也花分鐘數 (輸出 token)

通話品質 = 選什麼方案 (選什麼模型)

你的 Token 花在哪裡？

每次跟 AI 對話，Token 都在燒

一般對話

效率高



使用 MCP 工具的 Agent

效率低



下一代 Agent OS

最佳化



關鍵數字

MCP 協定可能讓 72% 的 Token 花在「告訴 AI 有哪些工具」，而不是「完成任務」

Phase 1

敢花 Token

黃仁勳的 50% 法則

「除了薪水之外，多給 50% 的 Token 預算，
換取 10 倍的生產力。」

— Jensen Huang, NVIDIA CEO

他在對誰說？

老闆

別省 AI 預算
先砸下去

員工

大膽使用
不要怕花 token

產業

AI 滲透率
還太低

Phase 1 — 敢花 Token

KPI：你用了多少 Token？

這個階段的重點

1

先用起來

多數人連 AI 都還沒開始用

2

量是關鍵

用越多 = AI 滲透率越高

3

不要省

省 token = 等於不給員工電腦

生活比喻

就像早期的雲端

2015 年企業上雲：

「先上雲再說！」

花多少錢不重要

重要的是先跑起來

現在的 AI 也在同一個階段：

「先用 AI 再說！」

Phase 1 — 敢花 Token

你現在該做的事

訂閱一個 AI

ChatGPT Plus / Claude
Pro / Gemini Advanced
月費 = 一杯咖啡 $\times$ 4

每天都用

寫信、整理資料、找答案
不要想著「省著用」

嘗試不同任務

不只問問題
讓 AI 寫程式、做表格、
分析數據

Phase 1 的贏家 = 敢花 Token 的人。他們建立了 AI 習慣，也讓 AI 了解了他們。

Phase 2

會花 Token

從量變質的轉折點

2026 年 3 月：MCP 爭議爆發

Ask 2026 開發者大會

Perplexity CTO：「我們內部已棄用 MCP」

回歸傳統 API 與 CLI，更快更省

YC CEO

「我自創的 CLI 比 MCP 可靠 100 倍」

社群平台一陣牆倒眾人推

同時間

Google、AWS、OpenAI 持續投入 MCP

產業分歧，不是死亡

本質問題

不是「MCP 好不好」，是「Token 該花在描述工具，還是完成任務？」

Phase 2 — 會花 Token

KPI：同樣的 Token，完成多少有效任務？

Phase 1 — 敢花

KPI = Token 用量

用越多越好

採用率是關鍵

Phase 2 — 會花

KPI = Token 效率

每個 token 都要有產出

品質比數量重要

轉折比喻

Phase 1 = 「先上雲再說」 → Phase 2 = **FinOps**（每一塊雲端支出都要有 ROI）

Token 經濟會壓縮在 1-2 年內走完這個轉換

你在花錢聽 AI 念履歷表

MCP 時代的 Token 分配



你請了一位年薪百萬的顧問

開會的前 70% 時間他在念自己的履歷

Agent OS 時代

未來的 Token 分配



工具描述編譯進 runtime

Token 100% 用在理解任務和推理

Part 4

工具架構演進

CLI → API → MCP → Agent OS

Agent 工具架構：四個時代

時代	技術	Token 成本	特點
CLI	命令列工具	近乎 0	最快、本地執行
Web API	REST / Function Calling	Low	成熟生態、最小 schema
MCP	Model Context Protocol	HIGH	標準化、但吃 context
Agent OS	原生工具整合	Minimal	工具編譯進 runtime

不是誰取代誰，是**場景決定工具**：個人用 CLI、SaaS 用 API、平台用 MCP/Agent OS

歷史總是驚人地相似

	雲端運算	手機上網	Token 經濟
Phase 1	「先上雲再說」 花多少不重要	「吃到飽」 流量隨便用	「先用 AI 再說」 Token 用量是 KPI
Phase 2	FinOps 每塊支出要 ROI	分級方案 省流量、Wi-Fi 優先	TokenOps 每個 Token 要產出
轉折點	帳單爆炸	降速通知	Context window 塞滿

錢可以一直加，但 **Context Window** 有物理上限 — 塞滿了就是塞滿了，逼你提早最佳化
就像手機流量用完被降速 — 你不會等月底，你會馬上學會省著用

產業正在分裂成兩條路線

輕量派

代表：Perplexity、YC

CLI + API + Function Calling

快、省 token、工程簡單

適合：個人工具、SaaS、高頻任務

標準化派

代表：OpenAI、Google、AWS

MCP / A2A / Agent Protocol

標準化、平台化、企業級

適合：多 Agent 協作、企業整合

你不需要選邊站 — 就像你同時用 LINE（輕量）和 Email（標準化），場景決定工具

Part 5

用 Token 思維看四階段

同樣的路徑，不同的效率

同樣四個階段，加入 Token 思維



前兩步突破「敢花」— 先建立 AI 習慣、廣泛接觸
後兩步進入「會花」— 知道什麼值得自動化、什麼不值得
同樣的四個階段，Token 思維讓你在每一步都更有效率

四階段 × Token 思維

階段	Token 心態	行為	關鍵詞
用工具	敢花 Token	每天問 AI 各種問題 什麼都試、不要省	廣撒網
選工具		每個 AI 都體驗一遍 找到各工具的強項	多嘗試
創工具	會花 Token	只留真正需要的工具 避免 MCP 式浪費	少即是多
改流程		只在高 ROI 環節用 AI 人做人擅長的事	精準部署

實際例子：同一件事，兩種花法

Phase 1 做法

任務：查今天台北天氣

裝好天氣 MCP Server

→ AI 載入 MCP schema (800 tokens)

→ 你問「台北天氣」(10 tokens)

→ AI 回答 (100 tokens)

總計：~910 tokens

Phase 2 做法

任務：查今天台北天氣

直接跟 AI 說「幫我查台北天氣」

→ AI 用內建搜尋 (0 schema)

→ AI 回答 (100 tokens)

總計：~110 tokens

8 倍效率差距 — 同樣的結果，Phase 2 省了 88% 的 Token。乘以一天 100 次查詢 = 巨大差異

你今天能做的三件事

1 Phase 1：先用起來（如果你還沒開始）

訂閱一個 AI、每天至少用 5 次、不要省

2 Phase 2：檢查你的 Token 花在哪裡

打開 AI 的用量設定，看看每次對話消耗多少 token

思考：「這些 token 有產出嗎？」

3 建立你的判斷框架

每次要用 AI 前問自己：

「這個任務值得花多少 token？有沒有更精簡的做法？」

一句話記住這堂課

Phase 1 的贏家是
「敢花 Token 的人」

Phase 2 的贏家是
「會花 Token 的人」

你現在在哪一個 Phase ?